



OWASP Top 10:2025 summary

LearnHub course reference: Web Application Security Basics

The OWASP Top 10 is an awareness document listing the most critical security risks to web applications. This page summarises each 2025 category in plain language with a typical example and a defence you can apply in an ASP.NET Core application.

ID	Category	Typical example	Defence
A01	Broken Access Control	A student changes an id in the URL and opens another student's quiz result.	Check authorisation and record ownership on the server for every request.
A02	Security Misconfiguration	Detailed error pages or default accounts left enabled in production.	Use environment-specific settings, friendly error pages and security headers.
A03	Software Supply Chain Failures	An outdated or compromised package is included in the build.	Pin versions, update dependencies and only use trusted package sources.
A04	Cryptographic Failures	Passwords stored in plain text or data sent over HTTP.	Use HTTPS with HSTS and a slow, salted password hash (ASP.NET Core Identity).
A05	Injection	User input is concatenated into a SQL query or written into a page as HTML.	Parameterised queries (EF Core), output encoding (Razor) and input validation.
A06	Insecure Design	No limit on login attempts, so passwords can be guessed.	Threat-model features and design controls such as lockout and rate limiting.
A07	Authentication Failures	Weak passwords accepted or session cookies readable by scripts.	Enforce a password policy, HttpOnly and Secure cookies, and account lockout.
A08	Software or Data Integrity Failures	Scripts loaded from an untrusted source or unsigned updates applied.	Serve scripts yourself or use integrity checks, and protect the deployment pipeline.
A09	Security Logging and Alerting Failures	Repeated failed logins go unnoticed.	Log security events without secrets and review or alert on them.
A10	Mishandling of Exceptional Conditions	An error path skips a permission check or reveals internal details.	Handle errors centrally, fail safely and never show stack traces to users.

Three habits that stop most attacks

- Validate every input on the server, even when the browser validates it too.
- Encode every output for its context: HTML, attributes, URLs and SQL parameters.
- Deny by default: check who the user is and which record they may touch.

ASP.NET Core features

- `[Authorize(Roles = "Admin")]` for role-based access.
- `AutoValidateAntiforgeryTokenAttribute` against CSRF.
- Identity lockout and the rate limiting middleware against brute force.